

# ISCHLSTROM Batteriemanagement (IBM)

Gemeinschaftsdienliche Steuerung privater Heimspeicher  
in einer Erneuerbare-Energie-Gemeinschaft

Energiegemeinschaft ISCHLSTROM  
[ischlstrom.org](http://ischlstrom.org)

August 2026

## Zusammenfassung

Das ISCHLSTROM Batteriemanagement (IBM) steuert private Heimspeicher von Mitgliedern einer Erneuerbare-Energie-Gemeinschaft (EEG) so, dass sie der Gemeinschaft nützen, ohne dem einzelnen Haushalt zu schaden. Zwei Eingriffe genügen dafür: An sonnigen Tagen regelt ein geschlossener Regelkreis die Ladeleistung der Batterie so, dass sie den ganzen Tag gerade schnell genug lädt, um am Abend voll zu sein; der restliche PV-Überschuss fließt laufend in die Gemeinschaft, solange diese ihn braucht. In der Nacht speist die Batterie kontrolliert in das Netz ein, solange die Gemeinschaft mehr verbraucht als erzeugt. Beide Eingriffe sind an Prognosen gebunden und mehrfach abgesichert: Ohne verlässliche Daten passiert nichts, und der Wechselrichter fällt nach wenigen Minuten von selbst in sein Werksverhalten zurück. Die Steuerung lernt Batteriekapazität, Ladeleistung und nächtliche Hauslast jeder Anlage selbstständig aus dem Betrieb; eine manuelle Parametrierung pro Anlage entfällt. Dieses Papier beschreibt Architektur, Algorithmen, Sicherheitsmechanismen und Betrieb des Systems im Detail.

## Inhaltsverzeichnis

|          |                                                                 |          |
|----------|-----------------------------------------------------------------|----------|
| <b>1</b> | <b>Motivation</b>                                               | <b>2</b> |
| <b>2</b> | <b>Systemarchitektur</b>                                        | <b>2</b> |
| 2.1      | Rollenteilung Server und Gateway . . . . .                      | 2        |
| 2.2      | Kern und Adapter . . . . .                                      | 3        |
| 2.3      | Zeitgesteuerte Regeln . . . . .                                 | 3        |
| <b>3</b> | <b>Steuersignale des Servers</b>                                | <b>3</b> |
| 3.1      | Datengrundlage: die Tagesprognose . . . . .                     | 4        |
| 3.2      | Community-Ladefenster . . . . .                                 | 4        |
| 3.3      | Individualisiertes Sperr-Ende . . . . .                         | 4        |
| 3.4      | Nachtbudget . . . . .                                           | 5        |
| 3.5      | Crossover-Zeiten . . . . .                                      | 5        |
| 3.6      | Wolkenvorschau . . . . .                                        | 5        |
| <b>4</b> | <b>Teil A: Laderegulation am Tag</b>                            | <b>5</b> |
| 4.1      | Grundprinzip: Regelkreis statt Sperrfenster . . . . .           | 5        |
| 4.2      | Umsetzung: direktes Limit oder Puls-Weiten-Modulation . . . . . | 6        |
| 4.3      | Rückfall: das Sperrfenster . . . . .                            | 6        |
| 4.4      | Wirkung für die Gemeinschaft . . . . .                          | 7        |

|          |                                                  |           |
|----------|--------------------------------------------------|-----------|
| <b>5</b> | <b>Teil B: Forcierte Entladung in der Nacht</b>  | <b>7</b>  |
| 5.1      | Entladefenster . . . . .                         | 7         |
| 5.2      | Entladeleistung aus der Wolkenvorschau . . . . . | 7         |
| 5.3      | Dynamische Leistungsgrenzen (C-Rate) . . . . .   | 7         |
| 5.4      | Untergrenzen: Reserve und Nachtbudget . . . . .  | 7         |
| <b>6</b> | <b>Selbstlernende Anlagenkennwerte</b>           | <b>8</b>  |
| 6.1      | Batteriekapazität . . . . .                      | 8         |
| 6.2      | Ladeleistung . . . . .                           | 8         |
| 6.3      | Nächtliche Hauslast . . . . .                    | 9         |
| <b>7</b> | <b>Sicherheit und Robustheit</b>                 | <b>9</b>  |
| <b>8</b> | <b>Betrieb und Überwachung</b>                   | <b>9</b>  |
| 8.1      | Status-Push und Vorstands-Dashboard . . . . .    | 9         |
| 8.2      | Authentifizierung . . . . .                      | 10        |
| 8.3      | Watchdog und Fernwartung . . . . .               | 10        |
| 8.4      | Installation . . . . .                           | 10        |
| <b>9</b> | <b>Zusammenfassung</b>                           | <b>10</b> |

# 1 Motivation

Eine Erneuerbare-Energie-Gemeinschaft verteilt lokal erzeugten PV-Strom unter ihren Mitgliedern. Ihr wirtschaftlicher und ökologischer Nutzen hängt davon ab, wie gut Erzeugung und Verbrauch zeitlich zusammenpassen: Nur Energie, die im selben Viertelstunden-Raster erzeugt und verbraucht wird, zählt als gemeinschaftliche Deckung. ISCHLSTROM kauft als Gemeinschaft niemals Strom zu; was die Mitglieder nicht aus der Gemeinschaft decken, beziehen sie individuell von ihrem eigenen Lieferanten.

Private Heimspeicher verschärfen das Gleichzeitigkeitsproblem, statt es zu lösen: Ab Sonnenaufgang lädt jede Batterie sofort, der Vormittags-Überschuss der PV-Anlagen verschwindet in den Speichern, während die Gemeinschaft noch Strom braucht. Am Abend und in der Nacht versorgt die volle Batterie nur den eigenen Haushalt, während die Gemeinschaft aus dem Netz bezieht.

IBM dreht dieses Verhalten um, ohne den Eigennutzen des Speicherbesitzers zu opfern:

- **Tagsüber (Laderegulung):** Die Batterie lädt so langsam wie möglich, wird aber bis zum Abend trotzdem voll. Der nicht gespeicherte Überschuss deckt die Verbrauchsspitzen der Gemeinschaft direkt, vom frühen Vormittag an und über den ganzen Tag verteilt.
- **Nacht (forcierte Entladung):** Die Batterie speist einen berechneten Teil ihres Inhalts in das Netz ein, solange die Gemeinschaft mehr verbraucht als erzeugt. Eingespeist wird nur, was der kommende Tag laut Prognose sicher wieder in die Batterie lädt.

Geladen wird dabei ausschließlich aus der eigenen PV-Anlage, nie aus dem Netz oder von anderen Mitgliedern.

## 2 Systemarchitektur

IBM besteht aus zwei Teilen: einem zentralen Server (die Website der Gemeinschaft, [ischlstrom.org](https://ischlstrom.org)) und je Anlage einem lokalen Gateway, einem Raspberry Pi mit openHAB (openHABian), der den Wechselrichter im Heimnetz des Mitglieds ansteuert.

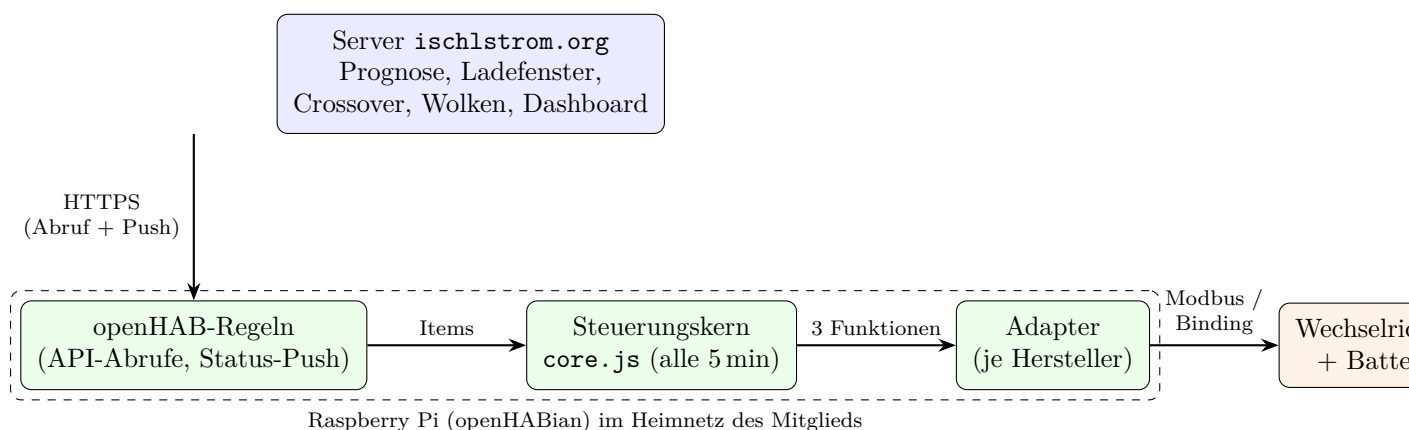


Abbildung 1: Gesamtarchitektur: Der Server liefert Prognosedaten, das lokale Gateway entscheidet und steuert. Alle Verbindungen gehen vom Gateway aus; am Router des Mitglieds wird nichts geöffnet.

### 2.1 Rollenteilung Server und Gateway

Der Server kennt die Gemeinschaft: die Viertelstunden-Prognose von Erzeugung und Verbrauch, die Wetterdaten und die gemeldeten Kennwerte aller Anlagen. Daraus berechnet er die Steuer-

signale (Abschnitt 3). Das Gateway kennt die Anlage: den Ladestand, den Wechselrichter und die lokal gelernten Kennwerte. Die eigentliche Entscheidung, ob in diesem Moment gesperrt oder entladen wird, fällt immer lokal auf dem Gateway. Fällt der Server aus, passiert nichts Gefährliches: Ohne aktuelle Daten unterbleiben die Eingriffe (Abschnitt 7).

## 2.2 Kern und Adapter

Die gesamte Entscheidungslogik liegt herstellerneutral in einem einzigen Skript (`control/core.js`), das alle fünf Minuten läuft und pro Kunde nie verändert wird. Alles Anlagenspezifische kommt aus openHAB-Items mit Rückfallwerten im Code; fehlt ein Item, läuft die Steuerung mit der Vorgabe weiter.

Herstellerspezifisch ist nur ein dünner *Adapter*, den das Setup dem Kern im selben Regel-Body voranstellt. Der Adapter-Kontrakt umfasst drei Pflichtfunktionen und eine optionale:

| Funktion                               | Semantik                                              | Rückgabe                     |
|----------------------------------------|-------------------------------------------------------|------------------------------|
| <code>ibmReset()</code>                | Wechselrichter sofort auf Werksverhalten              | <code>{ok}</code>            |
| <code>ibmPreventCharge(min)</code>     | Batterieladen für <code>min</code> Minuten sperren    | <code>{ok}</code>            |
| <code>ibmForceDischarge(W, min)</code> | Entladung mit $\approx W$ Watt erzwingen              | <code>{ok, appliedW?}</code> |
| <code>ibmLimitCharge(W, min)</code>    | optional: Ladeleistung auf $\approx W$ Watt begrenzen | <code>{ok, appliedW?}</code> |

Jede Aktion gilt nur für einen Zeitschlitz von fünf Minuten und muss herstellerseitig von selbst ablaufen (Schedule oder Revert-Timeout). Der nächste Lauf des Kerns verlängert den Eingriff oder beendet ihn, indem er schlicht nichts mehr kommandiert. `appliedW` meldet die nach herstellerseitiger Quantisierung tatsächlich kommandierte Leistung (etwa durch Prozent-Rundung); der Kapazitätsschätzer rechnet damit. `ibmLimitCharge` definieren nur Profile, deren Hardware ein echtes Ladelimit kennt, ohne dabei aus dem Netz laden zu können (SunSpec `InWRte`, Victron DVCC); fehlt die Funktion, bildet der Kern die Begrenzung per Puls-Weiten-Modulation über `ibmPreventCharge` nach (Abschnitt 4).

Unterstützt werden derzeit fünf Profile: Fronius GEN24 (Binding-Actions), Fronius Symo Hybrid (Modbus, SunSpec Model 124), Sigenergy SigenStor (proprietäre Modbus-Register), Deye SG04/SG05LP3 (Modbus RTU hinter einem RS485-Gateway, TOU-Fahrplan) und Victron Energy (Modbus TCP am GX-Gerät, ESS-Register). Ein neuer Hersteller braucht nur ein neues Profilverzeichnis mit Adapter; an den Setup-Skripten und am Kern ändert sich nichts.

## 2.3 Zeitgesteuerte Regeln

Auf dem Gateway laufen wenige, klar getrennte Regeln:

| Regel                            | Aufgabe                                     | Zeitplan      |
|----------------------------------|---------------------------------------------|---------------|
| <code>ibm_cloud_forecast</code>  | Wolkenvorschau abrufen                      | stündlich     |
| <code>ibm_crossover</code>       | Crossover-Zeiten abrufen                    | täglich 04:05 |
| <code>ibm_ladesperre</code>      | Ladefenster und Nachtbudget abrufen         | stündlich     |
| <code>ibm_battery_control</code> | Adapter + Kern: entscheiden und steuern     | alle 5 min    |
| <code>ibm_status_push</code>     | Zustand an das Dashboard melden             | alle 5 min    |
| <code>ibm_init</code>            | Startwerte setzen, solange Items NULL sind  | alle 10 min   |
| <code>ibm_pause</code>           | Pausentage herunterzählen                   | täglich 00:30 |
| <code>ibm_watchdog</code>        | Wechselrichter nach IP-Wechsel wiederfinden | bei Bedarf    |

## 3 Steuersignale des Servers

Der Server stellt vier Signale bereit. Die Gateways rufen sie über eine schlanke HTTP-API ab; die Community-Signale sind öffentlich, die individualisierten erfordern das Status-Token der Anlage.

| Endpunkt                    | Methode | Inhalt                               | Zugriff    |
|-----------------------------|---------|--------------------------------------|------------|
| /api/eeginfo/ladefenster/v1 | GET     | Community-Ladefenster                | öffentlich |
| /api/ibm/ladefenster/v1     | POST    | individuelles Fenster + Nachtbudget  | Token      |
| /api/eeginfo/crossover/v1   | GET     | Crossover-Zeiten der Woche           | öffentlich |
| /api/wolken/vorschau/v1     | GET     | Bewölkung im nächsten Mittagsfenster | öffentlich |
| /api/ibm/status/v1          | POST    | Status-Push der Anlage               | Token      |

### 3.1 Datengrundlage: die Tagesprognose

Grundlage der Ladefenster-Signale ist die Viertelstunden-Prognose der Gemeinschaft: ein statistisches Modell (Python) prognostiziert Erzeugung, Verbrauch und Eigendeckung je Messpunkt aus Kalender-, Sonnenstands- und Wetter-Merkmalen (Open-Meteo). Prognoseläufe werden versioniert gespeichert und nie überschrieben; die API liest stets den neuesten Live-Lauf. Die Wetterdaten selbst holt ein stündlicher Serverjob.

### 3.2 Community-Ladefenster

Aus dem Prognosetag berechnet der Server das Sperrfenster der Gemeinschaft. Mit der prognostizierten Erzeugung  $G_t$  und dem Verbrauch  $V_t$  je Viertelstunden-Slot, dem Überschuss  $S_t = G_t - V_t$  und dessen Tagesmaximum  $S_{\max}$  gilt:

- **Beginn** ist der erste Slot mit nennenswertem Sonnenschein,  $G_t > 0,05 \cdot G_{\max}$ .
- **Ende** ist der spätere Zeitpunkt von Vormittags-Crossover (erster Slot mit  $G_t \geq V_t$ ) und dem ersten Slot mit  $S_t \geq 0,75 \cdot S_{\max}$ , gekappt am Slot des Überschussmaximums und um 14:00:

$$t_{\text{Ende}} = \min\left(\max(t_{\text{Crossover}}, t_{S \geq 0,75 S_{\max}}), t_{S_{\max}}, 14:00\right). \quad (1)$$

Die Batterien bleiben so bis in die Überschusspitze gesperrt, statt direkt nach dem Crossover loszuladen. Erwartet die Prognose keinen Überschuss, liefert die API kein Ende und es wird nicht gesperrt. Jede Antwort trägt das Datum des Prognosetags; die Gateways verwerfen Fenster mit fremdem Datum.

### 3.3 Individualisiertes Sperr-Ende

Für Anlagen mit Status-Token rechnet der Server das Fensterende je Anlage, aus deren zuletzt gemeldeten Kennwerten (Kapazität  $C$ , Ladeleistung  $P_{\text{Lade}}$ , Abschnitt 6). Die Idee: Die erreichbare Ladeleistung folgt dem Erzeugungsprofil des Tages. Der Server normiert das Profil auf den Maximalslot des gesamten Prognoselaufs (nicht des Tages, sonst sähe ein trüber Tag wie ein klarer aus) und integriert rückwärts von der Abend-Deadline  $t_D$  (Ende des letzten Überschuss-Slots minus 60 Minuten):

$$E(t) = \sum_{s=t}^{t_D} P_{\text{Lade}} \cdot \frac{G_s}{G_{\text{Lauf}}^{\max}} \cdot 0,25 \text{ h}. \quad (2)$$

Das Sperr-Ende ist der späteste Slot  $t$ , ab dem  $E(t) \geq 0,95 \cdot C \cdot 1,3$  noch geladen wird (95% der Kapazität mit Sicherheitsfaktor 1,3), gekappt um 14:00. Reicht der ganze Tag nicht, wird nicht gesperrt. Individualisiert wird nur bei plausiblen Kennwerten (Kapazität 1 bis 100 kWh, Ladeleistung 0,3 bis 30 kW), und das Ende muss nach dem Fensterbeginn und nicht vor 05:00 liegen. Die Antwort ist als individuell markiert; das Gateway übernimmt sie dann unverändert und lässt die eigene Rechnung nach Gleichung (6) aus. Greift auf dem Gateway die Laderegelung (Abschnitt 4), spielt das Ende keine Rolle mehr; es bleibt der Rückfall für Anlagen ohne belastbare Schätzwerte.

### 3.4 Nachtbudget

Mit dem individualisierten Ladefenster liefert der Server das Entladebudget der kommenden Nacht. Er summiert zunächst, was die Anlage in den nächsten 24 Stunden laut Prognoseprofil laden kann (dieselbe Integration wie in Abschnitt 3.3, über alle Slots der nächsten 24 Stunden), zieht eine Eigenbedarfsreserve von 24 Stunden gelernter Hauslast  $P_{\text{Haus}}$  ab und behält einen Abschlag von 20% ein:

$$B = \max\left(0, E_{\text{ladbar},24\text{h}} - \frac{P_{\text{Haus}}}{1000} \cdot 24\text{h}\right) \cdot 0,8. \quad (3)$$

Meldet die Anlage keine plausible Hauslast (50 bis 3000 W), rechnet der Server mit 500 W. Bei mehrtägigem Schlechtwetter ist das Budget schlicht null: Die Batterie behält ihren Inhalt für das eigene Haus. Wie das Gateway das Budget in einen Ziel-Ladestand übersetzt, zeigt Gleichung (8).

### 3.5 Crossover-Zeiten

Die Crossover-Zeiten sind die typischen Uhrzeiten, zu denen die gemeinschaftliche Erzeugung den Gesamtverbrauch morgens über- und abends unterschreitet. Sie stammen aus den historischen Viertelstunden-Messdaten der Gemeinschaft: je Tag des Jahres über alle Jahre gemittelt und zu Kalenderwochen zusammengefasst (materialisierte Sicht, monatlich aufgefrischt). Anders als das Ladefenster sind sie also kein Prognoseprodukt, sondern ein saisonales Klimamittel; sie ändern sich langsam und sind deshalb als Wochenwert ausreichend genau.

### 3.6 Wolkenvorschau

Die Wolkenvorschau ist der mittlere Bewölkungsgrad über dem nächsten Mittagsfenster von 10:00 bis 14:00 (nach 12:00 zählt bereits das Fenster von morgen), berechnet aus den stündlich importierten Open-Meteo-Vorhersagedaten. Fehlen Wetterdaten, antwortet die API bewusst mit einem Fehler statt mit 0, denn 0% würde von den Gateways als voller Sonnenschein gelesen.

## 4 Teil A: Laderegung am Tag

### 4.1 Grundprinzip: Regelkreis statt Sperrfenster

An sonnigen Tagen wird die Ladeleistung der Batterie dynamisch geregelt. In jedem Fünf-Minuten-Zyklus berechnet der Kern die Ziel-Ladeleistung neu: fehlende Energie geteilt durch die verbleibende Zeit bis zur Abend-Deadline  $t_D$  (Abend-Crossover minus 60 Minuten), mit einem wolkenabhängigen Sicherheitsfaktor,

$$P_{\text{Soll}}(t) = \frac{(0,95 - \text{SoC}(t)) \cdot C}{t_D - t} \cdot \left(1,1 + \frac{w}{100} \cdot 0,5\right) \quad (4)$$

mit Kapazität  $C$  in kWh, Live-Ladestand  $\text{SoC}(t)$  als Anteil und Bewölkung  $w$  in Prozent: je bedeckter die (unter der Schwelle liegende) Vorschau, desto früher und schneller wird geladen. Die Batterie lädt so den ganzen Tag gerade schnell genug, um am Abend voll zu sein; der gesamte restliche PV-Überschuss fließt laufend in die Gemeinschaft. Weil auf den Live-Ladestand geregelt wird, ist der Kreis geschlossen: Zieht es zu, bleibt der Ladestand zurück,  $P_{\text{Soll}}$  steigt und die Begrenzung löst sich von selbst; wird es sonniger als vorhergesagt, sinkt  $P_{\text{Soll}}$  und mehr geht ins Netz. Prognosefehler regeln sich so alle fünf Minuten aus, statt einmal am Vormittag verwettet zu werden.

Geregelt wird nur zwischen dem Fensterbeginn des Servers (erster Sonnenschein der Tagesprognose, gültig nur für das mitgelieferte Datum) und der Abend-Deadline, und nur bei frischer, sonniger Wolkenvorschau: Liegt die Bewölkung über der Schwelle (Vorgabe 75%), bleibt das Laden frei, denn an einem trüben Tag würde die Batterie sonst womöglich gar nicht mehr voll.

## 4.2 Umsetzung: direktes Limit oder Puls-Weiten-Modulation

Definiert der Adapter `ibmLimitCharge`, wird  $P_{\text{Soll}}$  direkt kommandiert (quantisiert auf 100-W-Schritte). Sonst bildet der Kern die Begrenzung per Puls-Weiten-Modulation über `ibmPreventCharge` nach: Der Sperranteil

$$d = 1 - \frac{P_{\text{Soll}}}{P_{\text{Lade}}} \quad (5)$$

(mit der gelernten Ladeleistung  $P_{\text{Lade}}$ , Abschnitt 6) bestimmt, welcher Anteil der Zeit gesperrt wird. Entschieden wird je 15-Minuten-Block (drei Slots): Ein Bresenham-Akkumulator sammelt je Slot den Sperranteil als *Sperrschuld* an, jeder gesperrte Slot zieht 1 ab; im Mittel entsteht so genau die Ziel-Leistung. Eine Hysterese auf der Sperrschuld (Sperran ab 2,0, Freigeben unter 1,0) verhindert Flattern an den Blockgrenzen, und die Batterie schaltet nie schneller als im 15-Minuten-Raster. Zwei Deckel sichern die Regelung ab: Unter einem Sperranteil von 10% wird gar nicht begrenzt, und über 90% wird gekappt; ein Schätzfehler kann das Laden also nie ganz würgen. Fällt die Begrenzung ohnehin über die gelernte Ladeleistung hinaus ( $P_{\text{Soll}} \geq P_{\text{Lade}}$ ), lädt die Batterie unbegrenzt.

## 4.3 Rückfall: das Sperrfenster

Die Regelung braucht belastbare Schätzwerte für Kapazität und Ladeleistung. Solange sie fehlen (frisch installierte Anlage) oder die Regelung abgeschaltet ist, gilt das klassische Sperrfenster: Das Laden wird in Fünf-Minuten-Schritten komplett gesperrt, vom ersten Sonnenschein bis zu einem berechneten Sperr-Ende, danach lädt die Batterie mit voller Leistung. Für das Sperr-Ende gibt es drei Stufen, in absteigender Priorität:

1. **Individualisiertes Server-Ende:** Anlagen mit Status-Token rufen das Ladefenster über die Token-API ab. Der Server berechnet das Ende dann je Anlage aus dem Erzeugungsprofil des Prognosetags und den zuletzt gemeldeten Kennwerten (Kapazität, Ladeleistung): so spät, dass die Batterie am Abend gerade noch voll wird (Abschnitt 3.3). Die Antwort ist als individuell markiert; der Kern übernimmt sie unverändert.
2. **Lokal berechnetes Ende:** Ohne individualisierte Antwort rechnet der Kern selbst, sobald Kapazität und Ladeleistung gelernt sind (Abschnitt 6). Rückwärts vom Abend:

$$t_{\text{Ende}} = \min \left( t_{\text{Abend-Crossover}} - 60 \text{ min} - \underbrace{\frac{0,95 \cdot C}{P_{\text{Lade}}} \cdot 1,3 \cdot 60 \text{ min}}_{\text{Ladezeit mit Sicherheitsaufschlag}}, 13:00 \right) \quad (6)$$

mit Kapazität  $C$  in kWh und gelernter Ladeleistung  $P_{\text{Lade}}$  in kW. Als fehlende Energie gelten fest 95% der Kapazität, nicht der Live-Ladestand: Das Ergebnis soll den ganzen Vormittag konstant sein und nicht pendeln, sobald das Laden den Ladestand hebt. Liegt das berechnete Ende vor dem Fensterbeginn, braucht die Anlage den ganzen Tag zum Laden und es wird gar nicht gesperrt.

3. **Community-Ende des Servers:** Solange die Schätzer noch nicht belastbar sind, gilt das für die ganze Gemeinschaft berechnete Fensterende unverändert.

Greift die Laderegelung, sind alle drei Stufen gegenstandslos: Der geschlossene Regelkreis reagiert auf den Live-Ladestand besser, als es jede Vorausberechnung eines Endes kann. Der Fensterbeginn (erster Sonnenschein) und die Datumsprüfung kommen in jedem Fall vom Server. Plausibilitätsfenster verwerfen Unsinniges: Der Beginn muss zwischen 4 und 12 Uhr liegen, das Ende zwischen 5 und 15 Uhr.

## 4.4 Wirkung für die Gemeinschaft

Die Verbrauchsspitzen der Gemeinschaft werden direkt aus der PV gedeckt, und die Einspeisung verteilt sich über den ganzen Tag statt erst nach einem Sperr-Ende einzusetzen. Das hält die Gemeinschaft nach dem Crossover im Plus, fängt Abregelungsverluste und verkürzt nebenbei die Standzeit der Batterie bei 100% Ladung.

# 5 Teil B: Forcierte Entladung in der Nacht

## 5.1 Entladefenster

Entladen wird vom abendlichen bis zum morgendlichen Crossover, also genau solange die Gemeinschaft mehr verbraucht als erzeugt. Die Crossover-Zeiten holt das Gateway täglich vom Server. Liegen keine plausiblen Zeiten vor (Server nie erreichbar gewesen oder Werte unbrauchbar), wird nicht entladen; ein Ersatzfenster gibt es bewusst nicht.

## 5.2 Entladeleistung aus der Wolkenvorschau

Die Entladeleistung hängt von der Bewölkungsvorschau für das nächste Sonnenfenster ab, linear interpoliert zwischen einem Minimum und einem Maximum:

$$P_{\text{Entladung}} = P_{\max} - \frac{w}{100} (P_{\max} - P_{\min}), \quad w = \text{Bewölkung in \%}. \quad (7)$$

Bei klarer Vorschau ( $w = 0$ ) wird mit  $P_{\max}$  entladen, weil die Batterie am nächsten Tag sicher wieder voll wird; bei trüber Vorschau entsprechend weniger. Zwei Sonderfälle:

- **Trüb-Stopp:** Liegt die Vorschau über der Wolkenschwelle (dieselbe wie bei der Ladesperre), wird gar nicht eingespeist. Über eine lange Nacht entlädt auch minimale Leistung die Batterie fast vollständig; am trüben Folgetag müsste das Mitglied dann selbst Strom zukaufen. Es gilt die Symmetrie: Nur wenn morgen früh gesperrt würde (sonnig), wird heute Nacht auch eingespeist.
- **Ausfall der Vorschau:** Ist die Wolkenvorschau älter als drei Stunden oder ungültig, wird konservativ so entladen, als wäre der nächste Tag komplett bewölkt, also mit  $P_{\min}$ . Die Batterie läuft bei einem API-Ausfall damit nie mit Maximalleistung leer.

## 5.3 Dynamische Leistungsgrenzen (C-Rate)

$P_{\min}$  und  $P_{\max}$  sind einstellbar (Vorgabe 1000 und 3000 W), werden aber ersetzt, sobald die Kapazitätsschätzung belastbar ist (Abschnitt 6): Dann gilt eine C-Raten-Spanne von 0,1 C bis 0,3 C. Eine 10-kWh-Batterie speist also mit 1000 bis 3000 W ein, eine 5-kWh-Batterie mit 500 bis 1500 W. Unabhängig von Einstellung und Schätzung gilt eine harte Sicherheits-Obergrenze von 5000 W, die nie überschritten wird.

## 5.4 Untergrenzen: Reserve und Nachtbudget

Zwei Grenzen schützen den Haushalt vor einer leeren Batterie:

- **Reserve:** Unter den eingestellten Mindest-Ladestand (`IBM_MIN_BATTERY_CHARGE`, plausibel zwischen 5 und 90%) wird nie forciert entladen.
- **Nachtbudget:** Die Token-API liefert mit dem Ladefenster ein Entladebudget  $B$  in kWh: so viel darf heute Nacht eingespeist werden, ohne dass die Batterie am Folgetag dem eigenen Haus fehlt. Beim ersten Entlade-Zyklus der Nacht wird daraus ein Ziel-Ladestand berechnet



und für die ganze Nacht festgehalten:

$$\text{SoC}_{\text{Ziel}} = \max\left(\text{SoC}_{\text{min}}, \text{SoC}_{\text{Abend}} - \frac{B}{C} \cdot 100\right). \quad (8)$$

Das Festhalten verhindert, dass ein später aktualisiertes Budget in derselben Nacht doppelt ausgegeben wird; eine Lücke von mehr als 60 Minuten gilt als neue Nacht. Ohne (aktuelles) Budget, etwa ohne Token oder bei veraltetem Abruf, gilt wie zuvor nur die Reserve.

Nach dem Entlade-Stopp versorgt der Wechselrichter das Haus weiter aus der Batterie, bis zu seiner eigenen Reserve von wenigen Prozent; genau dieser Abschnitt liefert die Hauslast-Messung (Abschnitt 6).

## 6 Selbstlernende Anlagenkennwerte

Bei der Einrichtung sind weder Batteriekapazität noch reale Ladeleistung noch nächtliche Hauslast bekannt, und niemand soll sie pro Anlage pflegen müssen. Die Steuerung schätzt alle drei Werte aus dem laufenden Betrieb. Alle Schätzer teilen dasselbe Muster: Messstrecken über mehrere Fünf-Minuten-Läufe, Plausibilitätsfenster je Stichprobe, Neuaufsetzen nach Lücken oder widersprüchlichen Ladestandsbewegungen, gleitende Mittelung und eine Mindestzahl von Stichproben, bevor der Wert verwendet wird. Der Zustand jedes Schätzers liegt als JSON in einem persistierten Item und überlebt Neustarts.

### 6.1 Batteriekapazität

Während der forcierten Entladung ist die Batterieleistung bekannt, denn sie wird kommandiert. Der Kern summiert die entnommene Energie über die Fünf-Minuten-Läufe auf; fällt der Ladestand um mindestens 8 Prozentpunkte, ergibt sich eine Stichprobe

$$C_{\text{Stichprobe}} = \frac{E_{\text{entnommen}} [\text{kWh}]}{\Delta \text{SoC} [\%]} \cdot 100, \quad (9)$$

die mit Gewicht 0,3 gleitend in die Schätzung einfließt. Stichproben unter 1 oder über 100 kWh werden verworfen; verwendet wird die Schätzung ab drei akzeptierten Stichproben. Eine typische Nacht liefert mehrere Stichproben, die Schätzung ist also meist nach der ersten Nacht belastbar. Zieht der Haushalt nachts mehr als kommandiert, fällt die Schätzung etwas zu klein aus; das ist gewollt konservativ, denn die daraus abgeleitete Entladeleistung ist dann eher zu niedrig als zu hoch.

### 6.2 Ladeleistung

Gemessen wird nur, wenn die Messung die real erreichbare Ladeleistung widerspiegelt: tagsüber zwischen Fensterbeginn und Abend-Deadline, wenn die Batterie frei laden darf, die Vorschau Sonne meldet und der Ladestand unter 95% liegt (darüber drosselt der Wechselrichter selbst). In freien PWM-Blöcken der Laderegulation wird weiter gemessen (dort lädt die Batterie unbegrenzt); unter einem direkten Ladelimit nie, denn die Stichprobe würde das Limit statt der Anlage messen und die Schätzung nach unten ziehen. Steigt der Ladestand um mindestens 5 Prozentpunkte, ergibt geladene Energie (aus der Kapazität) geteilt durch die Messdauer eine Stichprobe in kW.

Die Mittelung ist bewusst *asymmetrisch und dauergewichtet*: Eine Stichprobe unter der bisherigen Schätzung (etwa Dunst trotz sonniger Vorschau) zählt mit Gewicht 0,5 je Messstunde, eine darüber nur mit 0,15 je Messstunde, gedeckelt bei 0,6. Die Schätzung liegt damit nahe an der unteren Hüllkurve der letzten Tage. Das ist die sichere Seite: Die Batterie muss abends voll sein, also wird die Ladezeit eher über- als unterschätzt und die Sperre endet eher zu früh als zu spät. Die Dauergewichtung verhindert zugleich, dass schnelle Mittagsphasen mit vielen Stichproben je Stunde den Schnitt systematisch nach oben ziehen.

## 6.3 Nächtliche Hauslast

Nach dem Entlade-Stopp versorgt die Batterie allein das Haus. Der Ladestandsabfall klar unterhalb der Entlade-Reserve (und oberhalb der wechselrichtereigenen Reserve) ergibt daher direkt die Hauslast: Stichproben zwischen 50 und 3000 W fließen mit Gewicht 0,3 in die Schätzung ein. Der Wert geht per Status-Push an den Server, der daraus die Eigenbedarfsreserve des Nachtbudgets ableitet; der Kreis zwischen lokalem Lernen und serverseitiger Individualisierung schließt sich hier.

## 7 Sicherheit und Robustheit

Das System greift in fremde Hardware ein und läuft unbeaufsichtigt in Privathaushalten. Entsprechend defensiv ist es ausgelegt; die Grundregel lautet: *Im Zweifel nichts tun*. Der Werkzustand des Wechselrichters, also Eigenverbrauchsoptimierung, ist immer der sichere Rückfall.

- **Fail-Safe im Wechselrichter:** Jeder Eingriff gilt für fünf Minuten und läuft herstellerseitig von selbst ab. Stirbt der Pi, hängt openHAB oder fällt das Netzwerk aus, arbeitet der Wechselrichter nach spätestens fünf Minuten wieder ab Werk. Kann ein Hersteller das nicht garantieren, dokumentiert sein Profil das Restrisiko ausdrücklich.
- **Kein Eingriff ohne Daten:** Ohne plausible Crossover-Zeiten keine Entladung; ohne gültiges, datumsrichtiges Ladefenster weder Laderegulation noch Sperre; ohne frische Wolkenvorschau (älter als drei Stunden) keine Ladebegrenzung und nur minimale Entladung.
- **Laden nie ganz würgen:** Die Laderegulation begrenzt höchstens auf 90% Sperranteil bzw. mindestens 10% der gelernten Ladeleistung. Selbst ein grober Schätzfehler lässt die Batterie weiter laden, und der Regelkreis korrigiert ihn über den Live-Ladestand.
- **Plausibilitätsfenster überall:** Jede Konfigurations- und Messgröße wird gegen einen erlaubten Bereich geprüft; Unplausibles wird verworfen und durch den Rückfallwert ersetzt, mit Logmeldung.
- **Harte Leistungsgrenze:** 5000 W Entladeleistung werden nie überschritten, weder durch Einstellungen noch durch die Kapazitätsschätzung.
- **Untergrenzen des Ladestands:** Reserve und Nachtbudget (Abschnitt 5) verhindern, dass der Haushalt mit leerer Batterie in einen trüben Tag startet.
- **Hauptschalter und Pause:** Der Hauptschalter steht nach der Installation auf **OFF**; erst das Mitglied schaltet die Steuerung scharf. Bei **ON** wird in jedem Zyklus zuerst `ibmReset()` gerufen, der Sollzustand ist also selbstheilend. Über die Pausenfunktion setzt das Mitglied die Steuerung für eine gewählte Zahl von Tagen aus (etwa bei Urlaub oder Handwerkerterminen); ein nächtlicher Zähler zählt herunter, danach läuft alles von selbst wieder an.
- **Robust gegen fehlende Konfiguration:** Jedes fehlende oder NULL-Item wird durch einen dokumentierten Rückfallwert ersetzt; die Steuerung läuft auch unvollständig eingerichtet weiter.

## 8 Betrieb und Überwachung

### 8.1 Status-Push und Vorstands-Dashboard

Auf Wunsch meldet jede Anlage alle fünf Minuten ihren Zustand an den Server: Ladestand, Wechselrichter-Status, Batterie-, Netz-, PV- und Lastleistung, alle Schalterstellungen, die gelernten Kennwerte, die zuletzt empfangenen Steuersignale, die Warnungen und Fehler aus dem

openHAB-Log der letzten 24 Stunden, Versionsstände (IBM-Paket, openHAB, Java, Betriebssystem), ausstehende Betriebssystem-Updates und den Systemzustand des Pi (CPU-Temperatur, Unterspannungs-Register, SD-Karten-Füllstand, RAM/Swap, Boot-Zeitpunkt). Übertragen werden ausschließlich diese Betriebsdaten, keine Verbrauchsdaten des Haushalts.

Der Vorstand sieht alle Anlagen live auf einem Dashboard (`/board/openhab`), inklusive Wochen-diagrammen von Ladestand, Batterieleistung und Netzeinspeisung aus der Batterie. Ausbleibende Meldungen färben die Anlage nach 15 Minuten gelb, nach einer Stunde rot; das Dashboard ist damit zugleich die Ausfallüberwachung der Flotte.

Die Netzeinspeisung aus der Batterie ist dabei eine berechnete Größe: Von der Entladeleistung fließt nur der Teil ins Netz, den das Netz gerade aufnimmt, der Rest versorgt den Haushalt:

$$P_{\text{Batterie} \rightarrow \text{Netz}} = \min(\max(P_{\text{Batterie}}, 0), \max(-P_{\text{Netz}}, 0)) \quad (10)$$

mit der Vorzeichenkonvention Batterie positiv beim Entladen, Netz negativ bei Einspeisung.

## 8.2 Authentifizierung

Jede Anlage authentifiziert sich mit einem Status-Token, das der Vorstand vor der Installation auf dem Dashboard je Mitglied erzeugt und das bei der Einrichtung in der Konfiguration des Gateways landet. Dasselbe Token individualisiert den Ladefenster-Abruf; beide Token-Endpunkte sind bewusst POST-Aufrufe mit dem Token im Body, damit es in keinem Access-Log auftaucht. Meldungen mit unbekanntem Token weist der Server ab; das Löschen des Tokens auf dem Dashboard widerruft den Zugang sofort. Auf dem Server ist der letzte gemeldete Zustand je Anlage gespeichert (begrenzt auf 16 kB), dazu eine Verlaufshistorie im Fünf-Minuten-Raster mit 30 Tagen Aufbewahrung, aus der die Dashboard-Diagramme entstehen; die Logauszüge gehen nicht in die Historie ein. Es werden keine Client-IPs und keine personenbezogenen Daten protokolliert.

## 8.3 Watchdog und Fernwartung

Vergibt der Router dem Wechselrichter per DHCP eine neue IP, findet ein Watchdog das Gerät im lokalen /24-Netz wieder, verifiziert es über die Seriennummer (nie wird ein fremdes Gerät übernommen) und trägt die neue Adresse per REST-API in das Thing ein. Für die Wartung hält jeder Pi einen ausgehenden WireGuard-Tunnel zum Wartungsserver offen; am Router des Mitglieds muss nichts geöffnet werden, und durch den Tunnel läuft ausschließlich das Wartungsnetz. Sicherheitsupdates des Betriebssystems spielt die Anlage über `unattended-upgrades` automatisch ein.

## 8.4 Installation

Die Einrichtung beim Mitglied besteht aus dem Flashen des openHABian-Images, dem Anlegen des Admin-Kontos und einem einzigen Kommando:

```
curl -fsSL https://ischlstrom.org/ibm/install.sh | sudo bash
```

Der Assistent findet den Wechselrichter im Netz, legt Things und Items an, installiert Regeln und Bedienoberfläche und verifiziert das Ergebnis. Alle Schritte sind idempotent; ein Paket-Update übernimmt die bestehende Konfiguration. Bedient wird die Anlage danach über die openHAB Main UI am Handy: Hauptschalter, Teilfunktionen, Pause und eine Expertenseite.

# 9 Zusammenfassung

IBM zeigt, dass sich private Heimspeicher mit geringem Aufwand gemeinschaftsdienlich steuern lassen, ohne Komfort- oder Kostennachteil für den Besitzer. Die Architektur trennt strikt: Der

Server rechnet mit dem Wissen der Gemeinschaft, das Gateway entscheidet mit dem Wissen der Anlage, und der Wechselrichter fällt ohne frische Kommandos von selbst in sein Werksverhalten zurück. Selbstlernende Schätzer für Kapazität, Ladeleistung und Hauslast ersetzen jede manuelle Parametrierung; das Kern-Adapter-Muster hält den Aufwand für neue Wechselrichter-Hersteller bei wenigen Dutzend Zeilen. Nach demselben Muster lässt sich das System auf weitere Gemeinschaften übertragen.